



Classification Problem

Logistic Regression Model

Thanh-Sach LE

✉ Itsach@hcmut.edu.vn

Faculty of Computer Science and Engineering
Ho Chi Minh City University of Technology

Ho Chi Minh City on February 5, 2026

Logistic Regression

Outline

Architecture View

Mathematical
View

Programming
View

Training methods

Outline

- ➊ Architecture View
- ➋ Mathematical View
- ➌ Programming View
- ➍ Training methods



Logistic Regression

1 Outline

Architecture View

Mathematical
View

Programming
View

Training methods



Logistic Regression

Outline

2 **Architecture View**

Mathematical
View

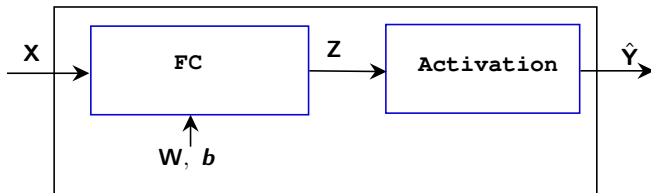
Programming
View

Training methods

Architecture View

Architecture View

Classification Problem



- N : number of samples (batch size)
- D : number of features (input dimension)
- K : number of classes (or outputs)
 - Binary: 2 classes, $K = 1$ output (one logit)
 - Single-label: $K > 1$ classes, K outputs (logits)
 - Multi-label: K labels, K outputs (one logit per label)



Logistic Regression

Outline

3 Architecture View

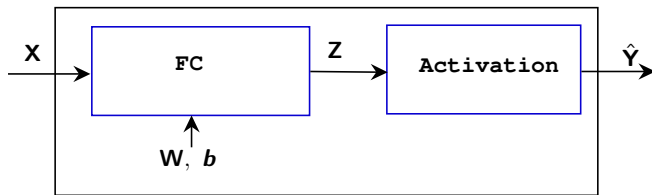
Mathematical View

Programming View

Training methods

Architecture View

Classification Problem



- **Convention:** vectors are column vectors; batch = stack $\mathbf{x}^T$ as rows; vectors = lowercase bold, matrices = uppercase bold.
- $\mathbf{X} \in \mathbb{R}^{B \times D}$: input (row n is $\mathbf{x}_n^T$)
- $\mathbf{Z} \in \mathbb{R}^{B \times K}$: FC output (logits)
- $\hat{\mathbf{Y}} \in \mathbb{R}^{B \times K}$: model output (probabilities)
- $\mathbf{Y}$: labels
- $\mathbf{W} \in \mathbb{R}^{D \times K}$, $\mathbf{b} \in \mathbb{R}^K$: parameters



Logistic Regression

Outline

4 Architecture View

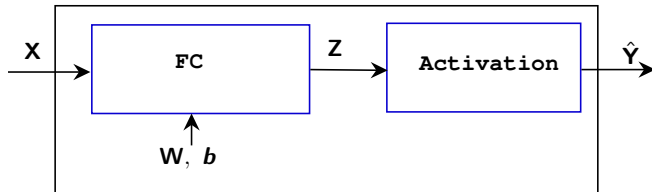
Mathematical View

Programming View

Training methods

Architecture View

Classification Problem



- FC (linear transformation):

$$\mathbf{Z} = \mathbf{X}\mathbf{W} + \mathbf{1}\mathbf{b}^T, \quad \mathbf{Z} \in \mathbb{R}^{B \times K}$$

- Activation (probabilistic interpretation):

$$\hat{\mathbf{Y}} = \text{Activation}(\mathbf{Z})$$



Logistic Regression

Outline

5 Architecture View

Mathematical View

Programming View

Training methods

Architecture View

Activation vs. Classification Type



Logistic Regression

Outline

6 Architecture View

Mathematical View

Programming View

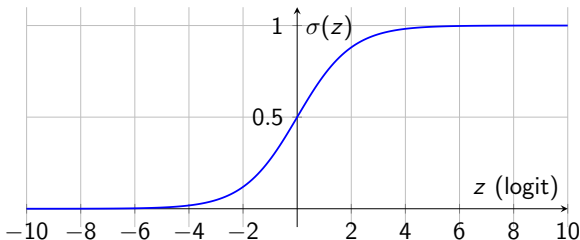
Training methods

Task	FC Output	Activation	Model Output
Binary Classification	$\mathbf{Z} \in \mathbb{R}^{B \times 1}$	Sigmoid	$\hat{\mathbf{Y}} = \sigma(\mathbf{Z}); \text{ scalar } \hat{y} = \sigma(z) = \frac{1}{1+e^{-z}}$
Single-label Classification	$\mathbf{Z} \in \mathbb{R}^{B \times K}$	Softmax	$\hat{Y}_{n,k} = \frac{e^{Z_{n,k}}}{\sum_{j=1}^K e^{Z_{n,j}}}$
Multi-label Classification	$\mathbf{Z} \in \mathbb{R}^{B \times K}$	Sigmoid (per class)	$\hat{Y}_{n,k} = \sigma(Z_{n,k}), \forall n, k$

- Activation determines the **probabilistic interpretation**
- Same architecture, different learning problems

Activation Function

Sigmoid: Logit $\rightarrow$ Probability



Sigmoid maps real-valued logits to probabilities in $(0, 1)$.

Definition

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Interpretation

- Input: **logit** $z = \mathbf{w}^T \mathbf{x} + b$
- Output: $\hat{y} \in (0, 1)$
- Interpreted as: $\hat{y} = P(y = 1 \mid \mathbf{x})$



Logistic Regression

Outline

7 Architecture View

Mathematical View

Programming View

Training methods

Activation Function

Softmax: Logits $\rightarrow$ Probability Distribution

Softmax Function

$$\hat{y}_k = (\text{softmax}(\mathbf{z}))_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad k = 1, \dots, K$$

Interpretation

- Input: $\mathbf{z} \in \mathbb{R}^K$ (logits)
- Output: $\hat{\mathbf{y}} \in \mathbb{R}^K$
- Properties:
 - $\hat{y}_k \geq 0$
 - $\sum_{k=1}^K \hat{y}_k = 1$
- Interpreted as a **categorical distribution**: $P(y = k \mid \mathbf{x}) = \hat{y}_k$

Logistic Regression

Outline

8

Architecture View

Mathematical View

Programming View

Training methods



Mathematical View

Logistic Regression

Outline

Architecture View

9 Mathematical View

Programming View

Training methods

Mathematical View

Single Sample

Linear Classification (single sample)

- For $\mathbf{x} \in \mathbb{R}^D$, the linear classifier computes logits:

$$\mathbf{z} = \mathbf{W}^T \mathbf{x} + \mathbf{b}$$

where $\mathbf{W} \in \mathbb{R}^{D \times K}$ and $\mathbf{b} \in \mathbb{R}^K$.

- Prediction (activation):

$$\hat{\mathbf{y}} = \text{Activation}(\mathbf{z})$$

Logistic regression (and softmax / multi-label logistic) are linear models followed by a non-linear activation.



Logistic Regression

Outline

Architecture View

10 Mathematical View

Programming View

Training methods

Mathematical View

Batch



Logistic Regression

Outline

Architecture View

11 Mathematical View

Programming View

Training methods

Linear Classification (batch)

- Batch input: $\mathbf{X} \in \mathbb{R}^{B \times D}$ (row n is $\mathbf{x}_n^T$)
- Logits and model output:

$$\mathbf{Z} = \mathbf{XW} + \mathbf{1b}^T, \quad \hat{\mathbf{Y}} = \text{Activation}(\mathbf{Z})$$



Programming View

Logistic Regression

Outline

Architecture View

Mathematical
View

12 Programming
View

Training methods

Programming View

PyTorch: Binary Classification



Logistic Regression

```
import torch
import torch.nn as nn

class BinaryClassifier(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.fc = nn.Linear(input_dim, 1)

    def forward(self, X): # X: (N, D)
        return self.fc(X) # logits

# Example
model = BinaryClassifier(input_dim=D)
```

Logistic Regression

Outline

Architecture View

Mathematical
View

13 Programming
View

Training methods

Loss: **BCEWithLogitsLoss** (*Sigmoid is applied internally*)

Programming View

PyTorch: Single-label Classification

Softmax Regression

```
import torch
import torch.nn as nn

class SoftmaxClassifier(nn.Module):
    def __init__(self, input_dim, num_classes):
        super().__init__()
        self.fc = nn.Linear(input_dim, num_classes)

    def forward(self, X): # X: (N, D)
        return self.fc(X) # logits

# Example
model = SoftmaxClassifier(input_dim=D, num_classes=K)
```

Loss: **CrossEntropyLoss** (Softmax included internally)



Logistic Regression

Outline

Architecture View

Mathematical
View

14 Programming
View

Training methods

Programming View

PyTorch: Multi-label Classification

Multi-label Logistic Regression

```
import torch
import torch.nn as nn

class MultiLabelClassifier(nn.Module):
    def __init__(self, input_dim, num_labels):
        super().__init__()
        self.fc = nn.Linear(input_dim, num_labels)

    def forward(self, X): # X: (N, D)
        return self.fc(X) # logits

# Example
model = MultiLabelClassifier(input_dim=D, num_labels=K)
```

Loss: **BCEWithLogitsLoss** (*Sigmoid applied internally, per label*)



Logistic Regression

Outline

Architecture View

Mathematical
View

15 Programming
View

Training methods

Programming View

PyTorch: Loss Function Notes



Logistic Regression

Outline

Architecture View

Mathematical View

16 Programming View

Training methods

Loss Functions vs. Classification Tasks

Task	Model output (logits)	Loss Function	Activation Inside Loss
Binary Classification	$\mathbf{Z} \in \mathbb{R}^{B \times 1}$	BCEWithLogitsLoss	Sigmoid
Single-label Classification	$\mathbf{Z} \in \mathbb{R}^{B \times K}$	CrossEntropyLoss	Softmax
Multi-label Classification	$\mathbf{Z} \in \mathbb{R}^{B \times K}$	BCEWithLogitsLoss	Sigmoid (per class)
Regression	$\hat{\mathbf{Y}} \in \mathbb{R}^{N \times K}$	MSELoss	None

Programming View

PyTorch: Loss Function Notes



Logistic Regression

Outline

Architecture View

Mathematical
View

17 Programming
View

Training methods

Important Note (PyTorch)

- **Do NOT add activation at the model output**
- Loss functions in PyTorch **already include** the required activation internally
- Adding activation twice leads to:
 - Wrong gradients
 - Slower or unstable training

Programming View

Keras: Classification Models

Binary / Multi-class / Multi-label

```
import tensorflow as tf
from tensorflow.keras import layers
# Binary classification
model_binary = tf.keras.Sequential([
    layers.Dense(1, activation="sigmoid", input_shape=(D,))
])
# Single-label classification
model_softmax = tf.keras.Sequential([
    layers.Dense(K, activation="softmax", input_shape=(D,))
])
# Multi-label classification
model_multilabel = tf.keras.Sequential([
    layers.Dense(K, activation="sigmoid", input_shape=(D,))
])
```



Logistic Regression

Outline

Architecture View

Mathematical
View

18 Programming
View

Training methods

Programming View

Keras: Loss Function Notes



Keras Design Philosophy

- In Keras, the **activation function is usually part of the model**
- By default, loss functions expect **probabilities** as input
- Therefore, model outputs are typically already normalized

Important Notes

- Forgetting the activation leads to invalid probability outputs
- Wrong activation $\Rightarrow$ wrong learning objective
- Some Keras losses support logits via `from_logits=True`

Logistic Regression

Outline

Architecture View

Mathematical View

19 Programming View

Training methods

Programming View

Keras: Loss Function Notes



Logistic Regression

Outline

Architecture View

Mathematical View

20

Programming View

Training methods

Activation vs. Loss in Keras

Task	Last Layer	Activation	Loss Function
Binary Classification	Dense(1)	Sigmoid	BinaryCrossentropy
Single-label Classification	Dense(K)	Softmax	CategoricalCrossentropy
Multi-label Classification	Dense(K)	Sigmoid	BinaryCrossentropy
Regression	Dense(K)	None (Linear)	MeanSquaredError



Training methods

Logistic Regression

Outline

Architecture View

Mathematical
View

Programming
View

21 Training methods

28

Training Methods

Overview



Logistic Regression

Outline

Architecture View

Mathematical View

Programming View

22

Training methods

Training Classification Models

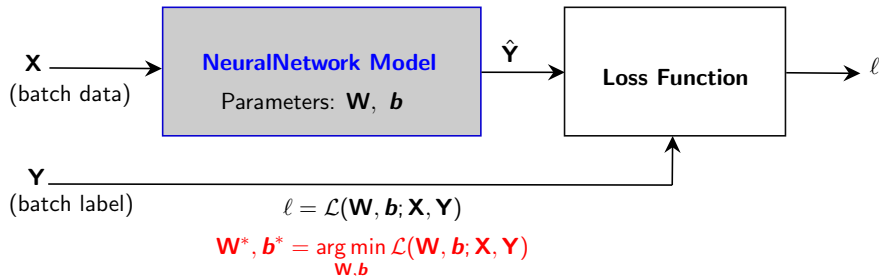
- **Closed-form Solution**
 - Not available for classification
 - Due to non-linear activation (Sigmoid / Softmax)
- **Gradient Descent (Iterative Optimization)**
 - Treat training as an optimization problem
 - Update parameters iteratively
 - Standard approach for all classification models

Classification models must be trained using iterative optimization.

28

Training Methods

Gradient Descent: Training Diagram



Logistic Regression

Outline

Architecture View

Mathematical View

Programming View

23 Training methods

Training Methods

Classification Model

Input, Output and Learnable Parameters

- Input:
 - Batch data: $\mathbf{X} \in \mathbb{R}^{B \times D}$
 - Batch label: $\mathbf{Y}$
- Model parameters:

$$\mathbf{W} \in \mathbb{R}^{D \times K}, \quad \mathbf{b} \in \mathbb{R}^K$$

- Model output (logits); batch: rows are $\mathbf{x}_n^T$

$$\mathbf{Z} = \mathbf{XW} + \mathbf{1b}^T$$

- Prediction (after activation):

$$\hat{\mathbf{Y}} = \text{Activation}(\mathbf{Z})$$



Logistic Regression

Outline

Architecture View

Mathematical View

Programming View

24

Training methods

28

Training Methods

Learning Objective



Logistic Regression

Outline

Architecture View

Mathematical
View

Programming
View

25

Training methods

Classification Loss Functions

- General optimization problem:

$$(\mathbf{W}^*, \mathbf{b}^*) = \arg \min_{\mathbf{W}, \mathbf{b}} \mathcal{L}(\hat{\mathbf{Y}}, \mathbf{Y})$$

- Choice of loss depends on classification type

28

Training Methods

Classification Losses

Common Classification Objectives

- **Binary Classification**

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

- **Single-label Classification (Softmax)**

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log \hat{y}_{ik}$$

- **Multi-label Classification**

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K \left[y_{ik} \log \hat{y}_{ik} + (1 - y_{ik}) \log(1 - \hat{y}_{ik}) \right]$$



Logistic Regression

Outline

Architecture View

Mathematical View

Programming View

26

Training methods

28

Training Methods

Gradient Descent for Classification



Logistic Regression

Outline

Architecture View

Mathematical
View

Programming
View

27 Training methods

Iterative Optimization

- Training as an optimization problem:

$$(\mathbf{W}^*, \mathbf{b}^*) = \arg \min_{\mathbf{W}, \mathbf{b}} \mathcal{L}(\hat{\mathbf{Y}}, \mathbf{Y})$$

- Parameter updates:

$$\mathbf{W}_{t+1} = \mathbf{W}_t - \eta \nabla_{\mathbf{W}} \mathcal{L}$$

$$\mathbf{b}_{t+1} = \mathbf{b}_t - \eta \nabla_{\mathbf{b}} \mathcal{L}$$

Training Methods

Key Takeaways

Summary

- Regression:
 - Can use closed-form or gradient descent
- Classification:
 - Requires non-linear activation
 - No closed-form solution
 - Must use gradient descent

Activation + Loss define the learning problem.



Logistic Regression

Outline

Architecture View

Mathematical View

Programming View

28 Training methods